ECHONET Lite Tutorial

Smart House Research Center, Kanagawa Institute of Technology

2019.11.11

Date	Description
2019.11.11	Alpha version release

CONTENTS

Abstract

This document is a technical Tutorial for the ECHONET Lite protocol. Understanding the "Basic level" will allow you to execute simple controls. Understanding the "Practical level" will allow you to understand most of ECHONET Lite. "Advanced level" content assumes product level. Chapter 4 simply explains the certification system, including AIF.

Prerequite

You are expected to have knowledge related to IP networks and UDP communication.

Terminology

Term	Description
HEMS	Home Energy Management System
Message	A cluster of data written in a certain format to be sent or received between devices. Depending on the meaning, it may be called a “request”, “response”, or “notification”.
Device	A Device that implements the ECHONET Lite protocol
EHD	ECHONET Lite Header
TID	Transaction Id
SEOJ	Source ECHONET Lite Object
DEOJ	Destination ECHONET Lite Object
ESV	ECHONET Lite Service
OPC	Operation Count
EPC	ECHONET Lite Property Code
PDC	Property Data Count
EDT	ECHONET Lite Data

1. Basic Level

1.1 Basic concept

- ECHONET Lite is a communication protocol between controllers and devices configured for HEMS.
- ECHONET Lite does not require a communication controller (coordinator) or devices to manage system configuration.
- Systems are usually configured with a controller and several devices. However, the controller and devices are equivalent in terms of protocol.
- ECHONET Lite is a protocol on the UDP layer.
  - Receiving port number: `3610`
  - Multicast address: `224.0.23.0`(IPv4), `ff02::1`(IPv6)
  - All messages are defined by binary data.
- The basic operations are `acquisition`, `setting`, and `notification` of properties.
  - Status and functions of the devices are defined as properties.
  - Get commands are used to acquire property values.
  - Set commands are used to set (operate) property values.
  - INF commands are used to notify property values.

1.2 Message frame format

ECHONET Lite messages are binary data. Message frame formats are shown below:

```
+---+---+----+----+---+---+----+----+----+---+----+----+----+
|EHD|TID|SEOJ|DEOJ|ESV|OPC|EPC1|PDC1|EDT1|...|EPCn|PDCn|EDTn|
+---+---+----+----+---+---+----+----+----+---+----+----+----+
```

Element	Data size (byte)	Description
EHD	2	ECHONET Lite Header (0x1081)
TID	2	Transaction Id
SEOJ	3	Source ECHONET Lite Object
DEOJ	3	Destination ECHONET Lite Object
ESV	1	ECHONET Lite Service
OPC	1	Operation Count
EPC	1	ECHONET Lite Property Code
PDC	1	Property Data Count
EDT	N	ECHONET Lite Data

In this chapter, “OPC=1” is used for simplicity. Message frame formats for this case are shown below:

```
+-----+-----+------+------+-----+----------+-----+-----+-----+
| EHD | TID | SEOJ | DEOJ | ESV | OPC=0x01 | EPC | PDC | EDT |
+-----+-----+------+------+-----+----------+-----+-----+-----+
```

EHD: ECHONET Lite Header

It is a fixed value (0x1081) that indicates an ECHONET Lite message.

TID: Transaction ID

This data corresponds to message requests and responses. Senders of requests shall send messages by setting any value as the message TID (normally, it is a value that increments per request sent). Receivers of the request shall send back messages by setting the TID value of the received message to the response message TID.

EOJ: ECHONET Lite Object

EOJ is 3-byte data that indicates the device of the sender and recipient. The sender shall be called SEOJ (Source EOJ), while the recipient is called DEOJ (Destination EOJ). The higher 2 bytes of EOJ indicate the type of device. Byte 3 of the EOJ is a code used to identify the device when multiple devices exist in the same category. Details are explained in 2.3 “EOJ”.

Examples of devices indicated by the higher 2 bytes of the EOJ are shown below:

EOJ (higher 2 bytes)	Device name
0x0130	Home Air Conditioner
0x027D	Storage Battery
0x0288	Low Voltage Smart Electric Energy Meter
0x0290	General Lighting
0x05FF	Controller
0x0EF0	Node profile

ESV: ECHONET Lite Service

ESV indicates the type of message service. The details are explained in 2.2 “ESV”. The following shows examples of ESV:

ESV	Symbol	Type	description
0x62	Get	request	Get property value
0x72	Get_Res	response	Response property value
0x61	SetC	request	Set property value
0x71	Set_Res	response	Ack to SetC command(*)
0x73	INF	notification	Inform property value

(*) Set_Res is a response indicating that it received SetC, instead of a response for executing a command. The Execute Get command is used to check whether the command has been executed or not.

OPC: Operation Count

An operation consists of an undermentioned set of EPC, PDC, and EDT. Multiple operations can be sent per message, and OPC shall indicate the number of operations in a message.

EPC: ECHONET Lite Property Code

A setting, a status or a function of a device is called “property”, and defined with 1-byte data.

PDC: Property Data Count

PDC indicates the undermentioned EDT data size. If ESV=0x62(Get), PDC becomes zero (0), since no EDT is required.

EDT: ECHONET Lite Data

The EDT is a property value. Length of data is variable.

Example of EPC and EDT for home air conditioner (EOJ=0x0130):

EPC	Property name	PDC	EDT
0x80	Operation status	1	0x30:ON, 0x31:OFF
0xA0	Air flow rate setting	1	8-step adjustment 0x31...0x38
0xB0	Operation mode setting	1	0x42:cooling, 0x43:heating
0xB3	Set temperature value	1	0 through 50
0xBB	Room temperature measurement	1	-127 ~ 125

Examples of EPC, PDC, EDT for general lighting (EOJ=0x0290)

EPC	Property name	PDC	EDT
0x80	Operation status	1	0x30:ON, 0x31:OFF
0xB0	Illuminance level setting	1	0 ~ 100%

EPC	Property name	PDC	EDT
0xB6	lighting mode setting	1	0x42:normal lighting, 0x43:night light, 0x45:colored lighting
0xC0	RGB setting	3	1 byte for each RGB, 0 ~ 255

1.3 Basic sequence

The basic sequence is explained while citing the operation status (EPC=0x80) of a home air conditioner as an example.

Acquiring a property value

Acquiring a property value use ESV=0x62(Get) as a request, and ESV=0x72(Get_Res) as a response. In the following example, you can see that the air conditioner is OFF(0x31).

```
Request: controller (0x05FF01) -> air conditioner (0x013001)
1081 0001 05FF01 013001 62  01  80  00
<--> <--> <----> <----> <-> <-> <-> <->
EHD  TID  SEOJ   DEOJ   ESV OPC EPC PDC

Response: air conditioner  (0x013001) -> controller  (0x05FF01)
1081 0001 013001 05FF01 72  01  80  01  31
<--> <--> <----> <----> <-> <-> <-> <-> <->
EHD  TID  SEOJ   DEOJ   ESV OPC EPC PDC EDT
```

Setting a property value

To set a property value, ESV=0x61(SetC) is used for a request, while ESV=0x71(SetC_Res) is used for a response. SetC_Res means “ACK” of receipt for the SetC command. Therefore, it is necessary to check separately to see if the property is actually set or not. The following is an example of turning air conditioner operation status ON (EDT=0x30).

```
Request: controller (0x05FF01) -> air conditioner (0x013001)
1081 0002 05FF01 013001 61  01  80  01  30
<--> <--> <----> <----> <-> <-> <-> <-> <->
EHD  TID  SEOJ   DEOJ   ESV OPC EPC PDC EDT

Response: air conditioner (0x013001) -> controller (0x05FF01)
1081 0002 013001 05FF01 71  01  80  00
<--> <--> <----> <----> <-> <-> <-> <->
EHD  TID  SEOJ   DEOJ   ESV OPC EPC PDC
```

Notifying a property value

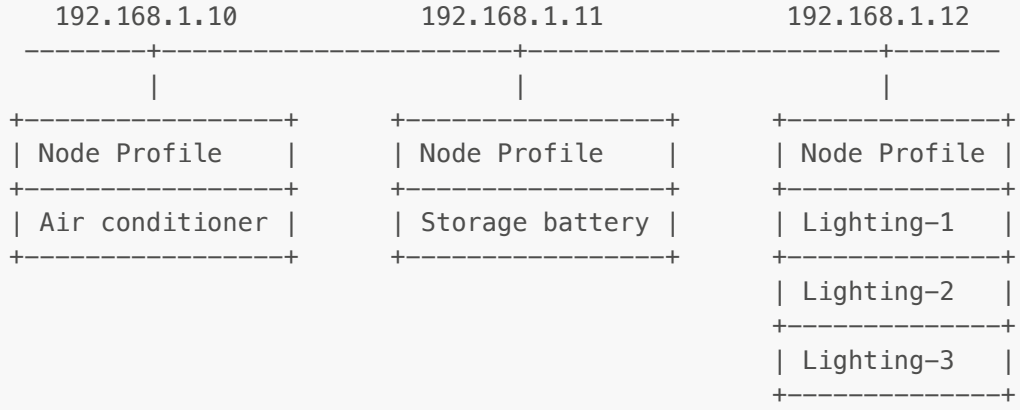
ESV=0x73(INF) is used for a property notification. The following is an example of a notification that the air conditioner operation status (EPC=0x80) turned ON(EDT=0x30).

```
Notification: air conditioner(0x013001) -> controller(0x05FF01)
1081 0001 013000 05FF01 73  01  80  01  30
```

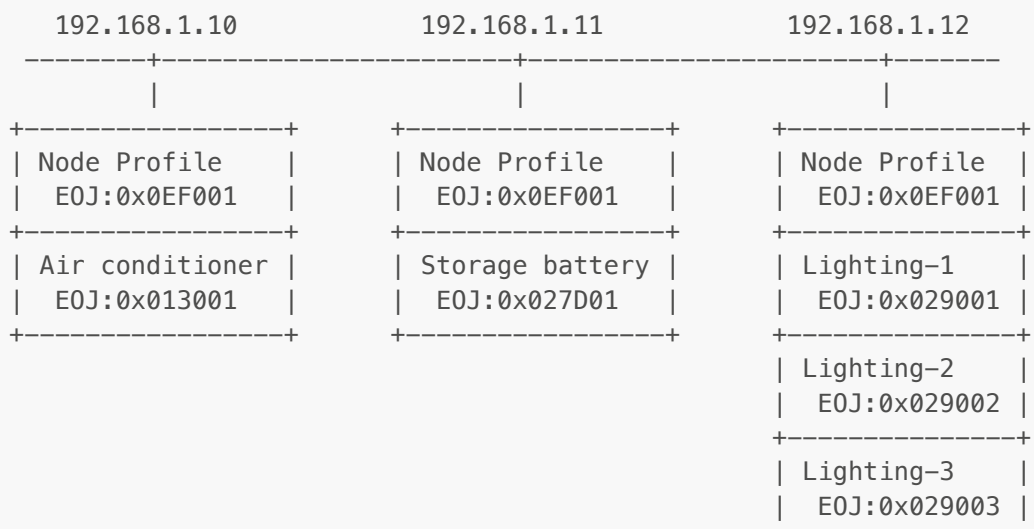
```
<--> <--> <----> <----> <-> <-> <-> <-> <->
EHD  TID  SEOJ   DEOJ   ESV OPC EPC PDC EDT
```

1.4 Node Profile Object and Device Search

To communicate through ECHONET Lite, it is necessary to know the IP address and EOJ of the receiving party. The following explains a method to check what kind of device object (EOJ) exists at which IP address within the network. An ECHONET Lite device consist of a node profile object that performs IP communication, and a device object(s) that realize the unique functions of the device. A node profile object exists corresponding to an IP address. Specific examples of device objects include home air conditioners, storage batteries, and general lighting. Configurations where multiple device objects exist at a single IP address are allowed.
The following shows examples of node profile object and device object configurations: A home air conditioner exists at the IP address 192.168.1.10, while a storage battery exists at the IP address 192.168.1.11, and three general lighting devices exist at the IP address 192.168.1.12.



A Sender (SEOJ) and a recipient (DEOJ) of ECHONET Lite communications shall designate the above-mentioned node profile object or device object. The EOJ consists of 3 bytes. The higher 2 bytes indicate the type of object (for example, 0x0EF0: Node Profile, 0x0130:home air conditioner, 0x027D:storage battery, 0x0290: general lighting), and byte 3 is a number for distinguishing objects in the same category. The EOJ of node profile object is always 0x0EF001. The following shows the examples of EOJ:



```
                                                    +---------------+
```

A Node profile object has device object information that exists at the IP address as a “self-node instance list S” (EPC=0xD6) property. The date format of this property is as follows:

> Byte 1: Number of device objects
> Byte 2 and later: A list of EOJs

For example, the values of self-node instance list S for each node profile object correspond to the above-mentioned IP addresses (192.168.1.10, 192.168.1.11, 192.168.1.12) are: “01013001”, “01027D01”, and “03028801028802028803”. Therefore, sending the message `“DEOJ=0x0EF001, ESV=0x62(Get), EPC=0xD6”` to devices in the network can reveal what kind of devices exist at which IP address.

The IP address of the receiving party remains unknown, because it is normally set with DHCP. Therefore, messages shall be sent to all devices in the network using UDP multicast. ECHONET Lite has registered 224.0.23.0 as a UDP multicast address.

This means that devices can be searched by **`multicasting the message “DEOJ=0x0EF001, ESV=0x62(Get), and EPC=0xD6” to IP=224.0.23.0`**.

The following shows an example of device search request and responses:

```
Request (Destination IP: 224.0.23.0)
1081 0001 05FF01 0EF001 62  01  D6  00
<--> <--> <----> <----> <-> <-> <-> <->
EHD  TID  SEOJ   DEOJ   ESV OPC EPC PDC

Response (Source IP: 192.168.1.10)
1081 0001 0EF001 05FF01 72  01  D6  04  01013001
Response (Source IP: 192.168.1.11)
1081 0001 0EF001 05FF01 72  01  D6  04  01027D01
Response (Source IP: 192.168.1.12)
1081 0001 0EF001 05FF01 72  01  D6  0A  03029001029002029003
<--> <--> <----> <----> <-> <-> <-> <-> <------------------>
EHD  TID  SEOJ   DEOJ   ESV OPC EPC PDC EDT
```

2. Practical Level

2.1 Specifications

ECHONET Lite specifications consist of following three documents. These can be downloaded from the ECHONET Consortium website.

- ECHONET Lite Specifications
- ECHONET Lite System Design Guidelines
- APPENDIX Detailed Requirements for ECHONET Device Objects

ECHONET Lite Specifications

The ECHONET Lite specifications are the main part of the specifications. The 300-page specification consists of Part 1 through Part 4. If you learn and understand this tutorial well, you won’t have to read the documentation for most cases. Refer to Part 2 as needed.

The latest version of the ECHONET Lite specifications as of August 2019 is Ver. 1.13. The ECHONET Lite version implemented on devices can be acquired as the version information property (EPC=0x82) of the node profile object (EOJ=0x0EF001). This tutorial is written on the basis of V1.13. Under normal device control, there is no need to pay special attention to the ECHONET Lite versions.

ECHONET Lite System Design Guidelines

The ECHONET Lite System Design Guidelines is an implementation guidelines. If you learn and understand this tutorial well, you won't have to read the documentation for most cases.

APPENDIX Detailed Requirements for ECHONET Device Objects

APPENDIX Detailed Requirements for ECHONET Device objects (hereinafter, shortened to "APPENDIX") has descriptions of property specifications for each device object. Refer this document to control devices. The specifications shall be revised about every six months, and the version is presented as a "Release". The Release starts with "A", and as of August 2019, Release "L" is the latest. Since property specifications do not care about backward compatibility(*), it is necessary for a controller to send messages according to the Release specifications implemented on the device or interpret received messages. The Release version implemented on a device can be acquired as the version information property (EPC=0x82) of the device object.

(*) Ex.: Degree-of-opening level (EPC=0xE1) of powered window shades (EOJ=0x0260) for Release A through C had eight steps (0x31 through 0x38). However, it was changed to numerical figures (0 through 100%) for Release D and subsequent versions.

JFYI, detailed information about properties of a node profile object is described in Part 2, 6.10 of the ECHONET Lite specifications.

The following indicates a part of the general lighting of APPENDIX Detailed Requirements for ECHONET Device objects. Property information is summarized in a table. Explanations of properties follow the table. Please note that additional information(Ex.: 0xFF is for undefined data) that in not in the table might be described there.

3．3．30 Requirements for general lighting class

Class group code : 0x02
Class code : 0x90
Instance code : 0x01–0x7F (0x00: All-instance specification code)

Property name	EPC	Contents of property Value range (decimal notation)	Data type	Data size	Unit	Access rule	Man-datory	Announce-ment at status change	Remark
Operation status	0x80	This property indicates the ON/OFF status. ON=0x30, OFF=0x31	unsigned char	1 byte	—	Set Get	○ ○	○	
Illuminance level	0xB0	This property indicates illuminance level in %. 0x00–0x64 (0–100%)	unsigned char	1 byte	%	Set/Get			

The following provides important notes for using this document (reference: Part 2, 6.2).

- If the value range is a numerical value
  - Values corresponding underflow and overflow are defined by data type and data size(*1)(reference: Part 2, 6.2.2).
  - Additional information (example: The corresponding value to the no set: 0xFF) may be added to the explanation for each property following the table.
- The access rule column indicates which service (ESV) corresponds. (Reference: Part 2, 6.2.5)

    - Set/ 0x60(SetI), 0x61(SetC), 0x6E(SetGet)
    - Get (Get) 0x62(Get), 0x6E(SetGet), 0x63(INF_REQ)
    - Anno 0x63(INF_REQ)
        - The only property whose access rule is “Anno” is EPC=0xD5 for the node profile.
- The mandatory column is for information related to implementation of services corresponding with Set or Get of the access rule. In case the access rule has a word of Set and/or Get, the mandatory column marked with “◯” means that their properties are always implemented. If the columns are blank, implementation of the property depends on the devices.
- When implementing properties whose "announcement at status change" column is marked with “◯”, INF or INFC shall be sent to multicast address (DEOJ=0x0EF001) when a property value is changed. (Reference: Part 2, 6.2.4)

(*1) Values corresponding with data type/data size, and underflow/overflow.

Data type	data size	underflow	overflow
signed char	1 byte	0x80	0x7F
signed short	2 byte	0x8000	0x7FFF
signed long	4 byte	0x80000000	0x7FFFFFFF
unsigned char	1 byte	0xFE	0xFF
unsigned short	2 byte	0xFFFE	0xFFFF
unsigned long	4 byte	0xFFFFFFFE	0xFFFFFFFF

2.2 ESV (ECHONET Lite service code)

A type of ECHONET Lite message is called a service, and it is defined with single-byte ESV (ECHONET Lite service code) data (reference: Part 2, 3.2.5, 4.2.3). Services can be categorized into the following four categories. For service implementation, see Chapter 3.

- Request: READ, WRITE, and notification request for a property value.
    - ESV=0x60, 0x61, 0x62, 0x63, 0x6E
- Response (normal): response (ESV=0x71, 0x72, 0x73, 0x7E) when requests are successfully accepted
- Response (abnormal): response when requests are not successfully accepted. This is called an Impossible response: SNA(*). (ESV=0x50, 0x51, 0x52, 0x53, 0x5E)
- Notification: Property value notification (ESV=0x73, 0x74)

(*) This is not clearly stated in the specifications, but it is assumed to be an abbreviation of “Service Not Available”.

Request ＆response

ESV Request	Symbol	Description	ESV Response (normal)	Symbol	ESV Response (abnormal)	Symbol	Note
0x60	SetI(*1)	Write (no response is required)	-	-	0x50	SetI_SNA	Not recommended
0x61	SetC(*2)	Write (response is required)	0x71	Set_Res(*3)	0x51	SetC_SNA	Frequently used

ESV Request	Symbol	Description	ESV Response (normal)	Symbol	ESV Response (abnormal)	Symbol	Note
0x62	Get	Read	0x72	Get_Res	0x52	Get_SNA	Frequently used
0x63	INF_REQ(*4)	Notification	0x73	INF(*5)	0x53	INF_SNA(*6)	Not recommended
0x6E	SetGet(*7)	Write&Read	0x7E	SetGet_Res	0x5E	SetGet_SNA	Not recommended

(*1) This is not clearly stated in the specifications, but it is supposed as an abbreviation of “Set with Ignorance”.
(*2) This is not clearly stated in the specifications, but it is assumed as an abbreviation of “Set with Confirmation”.
(*3) Set_Res means acknowledgment (ACK) of receipt for Set message.
(*4) INF_REQ should not be used with multicasting due to potential network congestion (reference: system design guideline 2.4.1).
(*5) INF as a response of INF_REQ is multicast (reference: Part 2, 4.2.3.5).
(*6) INF_SNA is unicast (reference: Part 2, 4.2.3.5).
(*7) Frame configuration of SetGet is different from the one explained in 1.2. See explanation in 3.1 “SetGet”.

- For Set, it is good enough to use SetC, so there is no need to actively use SetI.
- INF_REQ is a legacy command. No need to use it.
- The SetGet command design concept (reference: Part 2, 4.2.3.4) means sending multiple Set/Get commands in a single message. The order to execute the commands depends on the device, so it cannot be used for checking property values that are set. Also, error processing is complicated. Therefore, it is recommended to separately execute the Set and Get, rather than forcibly using the SetGet command.

Notification

ESV Notification	Symbol	Description	ESV response	Symbol	Note
0x73	INF	Notification (no response is required)	-	-	frequently used
0x74	INFC	Notification (response is required)	INFC_Res	0x7A	infrequently used unicast only

- In case of implementation of a property of which "Announcement on status change" is mandatory, INF command shall be sent with multicast. DEOJ is a node profile object(0x0EF001).
- INFC shall be used with unicast only (reference: Part 2, 4.2.3.6).

2.3 EOJ (ECHONET object)

The subject to send or receive ECHONET Lite messages is called the ECHONET object (EOJ). The EOJ consists of 3 bytes. Byte 1 is called a “class group code”, byte 2 is called a “class code” and byte 3 is called an “instance code” (reference: Part 2, 3.2.4). The class group code is defined as a broad category by device type (Ex. 0x01: air conditioner-related class group), the class code is defined as a detailed category (Ex. 0x30 for the air conditioner-related class group is for home air conditioners). However, for practical reasons, these two bytes represent device types, so that the higher 2 bytes of EOJ shall be called the device type in this document. The instance code is a code to identify the device when multiple devices exist under the same category, and the value is set by the device itself. The range of the value is 1 through 255.

- Device type: The value 0x0F00 through 0FFF is a code that users can freely define and use.

- The 0x00 for the instance code of DEOJ is a wildcard that sets all the devices in the same device type to the recipient. Taking DEOJ=0x029000 as an example, the same message can be sent to all general lighting devices at the recipient IP address.
- EOJ of a node profile object is 0x0EF001, except for a transmission-only node, that is 0x0EF002 (see 3.4 “Transmission-only node”).
- Instance code is set by the device itself, but there is no rules or guidelines about the value. In many cases, the value is 0x01 and if there are multiple devices with the same device type, it is incremented. However, you should not count on this assumption.

The following shows examples of device type:

Device type	Device name	Device type	Device name	Device type	Device name
0x000D	Illuminance sensor	0x0272	Instantaneous water heater	0x0288	Low voltage smart electric energy meter
0x0011	Temperature sensor	0x0273	Bathroom dryer	0x028A	High voltage smart electric energy meter
0x0012	Humidity sensor	0x0279	Household solar power generation	0x0290	General lighting class
0x0022	Electric energy sensor	0x027A	Cold/hot water heat source equipment	0x0291	Mono functional lighting
0x002D	Air pressure sensor	0x027B	Floor heater	0x02A1	Electric vehicle charger
0x0130	Home air conditioner	0x027C	Fuel cell	0x02A6	Hybrid water heater
0x0133	Ventilation fan	0x027D	Storage battery	0x03B7	Refrigerator
0x0135	Air cleaner	0x027E	Electric vehicle charger/discharger	0x03B8	Combination microwave oven (Electronic oven)
0x0260	Powered window shades	0x0280	Watt-hour meter	0x03B9	Cooking heater
0x0263	Electrically operated rain sliding door/shutter	0x0281	Water flow meter	0x03D3	Washer and dryer
0x026B	Electric water heater	0x0282	Gas meter	0x05FD	Switch
0x026F	Electric lock	0x0287	Distribution panel metering	0x05FF	Controller

2.4 EPC

2.4.1 EPCs of device objects

The properties of device objects are defined with 1-byte code called as EPC, and those consist of properties of Super class(reference:APPENDIX Chapter 2) and properties of device objects(reference:APPENDIX Chapter 3). Properties of Super class are identical through devices, and may be overridden with definitions of each device object.

The following shows properties of Super class. Please see APPENDIX Chapter 3 for properties of each device object.

Super class

EPC	Property name	access rule	mandatory	status change announcement	Note
0x80	Operation status	Set Get	x ○	○	
0x81	Installation location	Set/Get	○	○	(*1)
0x82	Standard version information	Get	○	x	(*2)
0x83	Identification number	Get	x	x	(*3)
0x84	Measured instantaneous power consumption	Get	x	x	
0x85	Measured cumulative power consumption	Get	x	x	
0x86	Manufacturer's fault code	Get	x	x	
0x87	Current limit setting	Set/Get	x	x	
0x88	Fault status	Get	○	○	
0x89	Fault description	Get	x	x	
0x8A	Manufacturer code	Get	○	x	
0x8B	Business facility code	Get	x	x	
0x8C	Product code	Get	x	x	
0x8D	Production number	Get	x	x	
0x8E	Production date	Get	x	x	
0x8F	Power-saving operation setting	Set/Get	x	x	
0x93	Remote control setting	Set/Get	x	x	(*4)
0x97	Current time setting	Set/Get	x	x	
0x98	Current date setting	Set/Get	x	x	
0x99	Power limit setting	Set/Get	x	x	
0x9A	Cumulative operating time	Get	x	x	
0x9D	Status change announcement property map	Get	○	x	
0x9E	Set property map	Get	○	x	
0x9F	Get property map	Get	○	x	

(*1) Property value is 1 byte or 17 bytes.
For 1 byte, bitmap (reference: APPENDIX2.2) 0x00: undefined, 0xFF: unknown.
(*2) APPENDIX Release version implemented
Byte 1 and 2:0x00 (except Release A).

Byte 3: Represents the Release version as ASCII code. Only A is a lowercase letter (0x61), while B and the following letters are uppercase letters (B:0x42...L: 0x4C).
4th byte:0x00
(*3) 0xFE + Manufacturer code + Unique ID (13 bytes)
(*4) See 3.3 “Remote control setting”

EPC values are categorized into the following by the scope of the value. (Reference: Part 2, 3.2.7)

EPC	Category	Example
0x00 ~ 0x7F	Reserved	
0x80 ~ 0x9F	Common properties across devices	0x80: Operation status
0xA0 ~ 0xEF	Specific properties to each device	0xB0 air conditioner: Operating mode Lighting: Illuminance level
0xF0 ~ 0xFF	User defined properties	

Properties of Super class are EPC=0x80~0x9F except followings. These are described in the specifications of each device object when necessary.

EPC	Property name
0x90	ON timer setting
0x91	ON timer time setting
0x92	ON timer relative time setting
0x94	OFF timer setting
0x95	OFF timer time setting
0x96	OFF timer relative time setting
0x9B	(reserved for legacy)
0x9C	(reserved for legacy)

2.4.2 EPCs of node profile object

The properties of Node profile object are defined with 1-byte code called as EPC, and those consist of properties of Super class(reference: Part 2, 6.10.1) and properties of Node profile object(reference: Part 2, 6.11.1). Super class of Node profile object is different from that of device objects. The following shows all properties of Node profile object including its Super class.

EPC	property name	access rule	mandatory	status change announcement	note
0x80	Operation status	Set Get	x ◯	◯	
0x82	Version information	Get	◯	x	(*1)
0x83	Identification number	Get	◯	x	(*2)
0x88	Fault status	Get	x	x	
0x89	Fault description	Get	x	x	

EPC	property name	access rule	mandatory	status change announcement	note
0x8A	Manufacturer code	Get	○	x	
0x8B	Business facility code	Get	x	x	
0x8C	Product code	Get	x	x	
0x8D	Production number	Get	x	x	
0x8E	Production date	Get	x	x	
0x9D	Status change announcement property map	Get	○	x	
0x9E	Set property map	Get	○	x	
0x9F	Get property map	Get	○	x	
0xBF	Unique identifier data	Set/Get	x	x	
0xD3	Number of self-node instances	Get	○	x	(*3)
0xD4	Number of self-node classess	Get	○	x	(*4)
0xD5	Instance list notification	Anno	○	○	(*5)
0xD6	Self-node instance list S	Get	○	x	(*5)
0xD7	Self-node instance list S	Get	○	x	(*3)

(*1) Version of ECHONET Lite standard.
Byte 1: major version
Byte 2: minor version
Byte 3: message format is shown in a form of bitmap
Byte 4: 0x00
Ex.: 0x010D0100: V1.13
(*2) 0xFE + Manufacturer code + Unique ID (13 bytes)
(*3) Not including node profile class
(*4) Including node profile class
(*5) EDT for EPC=0xD5 and 0xD6 are same.

The following shows differences of Super class between Node profile object and device objects.

EPC	Node profile object	Device object	Note
0x81	Not applicable	Installation location	
0x82	Version information	Standard version information	ECHONET Lite version: Node profile object Appendix version: Device objects
0x83	Identification number	Identification number	Access rule: Mandatory(Node profile object) Access rule: Not mandatory(Device object)
0x84	Not applicable	Measured instantaneous power consumption	

EPC	Node profile object	Device object	Note
0x85	Not applicable	Measured cumulative power consumption	
0x86	Not applicable	Manufacturer's fault code	
0x87	Not applicable	Current limit setting	
0x88	Fault status	Fault status	Access rule: Not mandatory(Node profile object) Access rule: Mandatory(Device object)
0x8F	Not applicable	Power-saving operation setting	
0x93	Not applicable	Remote controll setting	
0x97	Not applicable	Current time setting	
0x98	Not applicable	Current date setting	
0x99	Not applicable	Power limit setting	
0x9A	Not applicable	Cumulative operating time	

2.4.3 Atomic operation

In the information engineering world, combining multiple operations to make a single operation is called an "atomic operation". The following shows EPC that can be considered "atomic operations" in terms of ECHONET Lite. Before executing the 2nd EPC (Get), it is necessary to execute the 1st EPC (SetC).

EOJ	Device object name	1st EPC	property name	2nd EPC	property name
0x027E	EV charger/discharger	0xCD	Vehicle connection confirmation	0xC7	Vehicle connection and chargeable/dischargeable status
0x0287	Power distribution board	0xB2	Channel range specification for cumulative amount of electric power consumption measurement(simplex)	0xB3	Measured cumulative amount of electric power consumption list(simplex)
-	-	0xB4	Channel range specification for instataneous current measurement (simplex)	0xB5	Measured instantaneous current list (simplex)
-	-	0xB6	Channel range specification for instantaneous power consumption measurement(simplex)	0xB7	Measured instantaneous power consumption list(simplex)
-	-	0xB9	Channel range specification for cumulative amount of electric power consumption measurement(duplex)	0xBA	Measured cumulative amount of electric power consumption list(duplex)

EOJ	Device object name	1st EPC	property name	2nd EPC	property name
-	-	0xBB	Channel range specification for instantaneous current measurement(duplex)	0xBC	Measured instantaneous current list (duplex)
-	-	0xBD	Channel range specification for instantaneous power consumption measurement(duplex)	0xBE	Measured instantaneous power consumption list(duplex)
-	-	0xC5	Day for which the historical data of measured cumulative amounts of electric energy is to be retrieved	0xC3	Historical data of measured cumulative amounts of electric energy (normal direction)
0x0288	Low-voltage smart electric energy meter	0xE5	Day for which the historical data of measured cumulative amounts of electric energy is to be retrieved 1	0xE2	Historical data of measured cumulative amounts of electric energy 1 (normal direction)
-	-	0xE5	Day for which the historical data of measured cumulative amounts of electric energy is to be retrieved 1	0xE4	Historical data of measured cumulative amounts of electric energy 1 (reverse direction)
-	-	0xED	Day for which the historical data of measured cumulative amounts of electric energy is to be retrieved 2	0xEC	Historical data of measured cumulative amounts of electric energy 2 (normal and reverse directions)
0x028A	High-voltage electric energy meter	0xE1	Day for which the historical data of measured cumulative amounts of electric energy is to be retrieved	0xC6	Historical data of measured electric power demand
-	-	0xE1	Day for which the historical data of measured cumulative amounts of electric energy is to be retrieved	0xCE	Historical data of measurement data of cumulative amount of reactive electric power consumption (lag) for power factor measurement
-	-	0xE1	Day for which the historical data of measured cumulative amounts of electric energy is to be retrieved	0xE7	Historical data of measured cumulative amount of active electric energy
0x02A1	EV charger	0xCD	Vehicle connection confirmation	0xC7	Vehicle connection and chargeable status

2.5 EDT

The value of EDT is binary data. It is up to EPC how it is interpreted. The specifications are described in the APPENDIX as contents of property, value range, data type, data size and unit. Three data types, number,

state(status) and level are common.

number

In case of number, data size and signed/unsigned information are described by the key words of C language. The following is an example of temperature setting(EPC=0xB3) of air conditioner(EOJ=0x0130).

Set temperature value	0xB3	Used to set the temperature and to acquire the current setting. 0x00–0x32 (0–50°C)	unsigned char	1 byte	°C	Set/Get	○		

In case the number includes decimal place, a coefficient is described in the unit column. For example of relative temperature setting(EPC=0xBF) of air conditioner, if the value of EDT is 25, it means 2.5°C.

Relative temperature setting	0xBF	Used to set the relative temperature relative to the target temperature for an air conditioner operation mode, and to acquire the current setting. 0x81–0x7D (-12.7°C–12.5°C)	unsigned char	1 byte	0.1 °C	Set/Get			

The values for underflow and overflow are defined for each data type. (reference:第２部6.2.2)。For example, when the fetched data type is unsigned char and its value is 0xFF(255), the internal value of the device is equal or more than 254.

data type	data size	underflow	overflow
signed char	1 byte	0x80	0x7F
signed short	2 byte	0x8000	0x7FFF
signed long	4 byte	0x80000000	0x7FFFFFFF
unsigned char	1 byte	0xFE	0xFF
unsigned short	2 byte	0xFFFE	0xFFFF
unsigned long	4 byte	0xFFFFFFFE	0xFFFFFFFF

state(status)

By assigning a specific value to each state(status) such as ON/OFF of power or cooling/heating of operation mode setting, EDT can handle state(status). The following is an example of operation mode setting(EPC=0xB0) of air conditioner(0x0130).

Operation mode setting	0xB0	Used to specify the operation mode (“automatic,” “cooling,” “heating,” “dehumidification,” “air circulator” or “other”), and to acquire the current setting. The following values shall be used: Automatic: 0x41 Cooling: 0x42 Heating: 0x43 Dehumidification: 0x44 Air circulator: 0x45 Other: 0x40	unsigned char	1 byte	–	Set/Get	○	○	

level

By assigning a specific value to each level such as air flow rate setting of air conditioner, EDT can handle level. The following is an example of air flow rate setting(EPC=0xA0) of air conditioner(0x0130).

Air flow rate setting	0xA0	Used to specify the air flow rate or use the function to automatically control the air flow rate, and to acquire the current setting. The air flow rate shall be selected from among the 8 predefined levels. Automatic air flow rate control function used = 0x41 Air flow rate = 0x31–0x38	unsigned char	1 byte	–	Set/Get	○	○	

Plural data

Plural data can be packed into EDT. The following of RGB setting(EPC=0xC0) of lighting(EOJ=0x0290) shows that the first one byte data is for R, second one is for G and thrd one is for B.

RGB setting for color lighting	0xC0	Used to set the RGB value for color lighting and to acquire the current setting. Byte 1: R Byte 2: G Byte 3: B 0x00–0xFF (0–255) Minimum brightness=0x00, maximum brightness=0xFF	unsigned char × 3	3 bytes	—	Set/ Get			

The following of historical data(EPC=0xE2) of smart meter(EOJ=0x0288) shows that the first two bytes data is for days and 48 sets of 4 byte data are for electric power data.

Historical data of measured cumulative amounts of electric energy 1 (normal direction)	0xE2	This property indicates the day for which the historical data of measured cumulative amounts of electric energy is to be retrieved 1 and the historical data of measured cumulative amounts of electric energy (normal direction), which consists of 48 items of half-hourly data for the preceding 24 hours (00:00 to 23:30) of the day by time series from the highest-order byte. 1-2 bytes: day for which the historical data of measured cumulative amounts of electric energy is to be retrieved 0x0000–0x0063 (0-99) 3 and succeeding bytes: measured cumulative amounts of electric energy 0x00000000–0x05F5E0FF (0–99,999,999)	unsigned short + unsigned long × 48	194 bytes	kWh	Get	○		

2.6 Configurations of a Node profile object and device objects

Configurations of a Node profile object and device objects of the ECHONET Lite devices may include the following three types.

```
 +---+  +-----+      |         |
                   +---+     +---+
                   | D | . . | D |
                   +---+     +---+

N:Node profile object, D:Device object
```

(A) Basic configuration: In case products consist of a Node profile and a device object
(B) In case a product consists of a node profile and two or more device objects
Ex.: A floor heater consists of a floor heater object and water heat source equipment object.
(C) When two or more independent products share a single Node profile object. Communications such as low-power radios, Bluetooth, and WiSUN-HAN can be used for connections between the Node profile object and device objects.
Ex.: Panasonic AiSEG, Toshiba Feminity
For configurations of (B) and (C), two or more device object EOJs shall be described in the Node profile instance list, EPC=0xD5 and 0xD6.

2.7 Startup processes

2.7.1 Device

Devices notify of the participation of new devices in the network using a multicasting instance list (recipient IP=224.0.23.0).

```
Destination IP: 224.0.23.0
SEOJ: 0x0EF001
DEOJ: 0x0EF001
ESV:  0x73(INF)
OPC:  1
EPC:  0xD5
EDT:  Instance list within own node
```

Ex.: Home air conditioner

```
Controller                                  Home air conditioner
(IP=192.168.1.1)                            (IP=192.168.1.2)
(EOJ=0x05FF01)                              (EOJ=0x013001)
    |  IP SRC:192.168.1.2, DST:224.0.23.0          |
    |  1081 0001 0EF001 0EF001 73 01 D5 04 01013001 |
    | <-------------------------------------------- |
    |                                               |
```

2.7.2 Controller

A controllers search devices during startup and then acquire the information necessary for operation from devices. Here, at least the required operations are explained.

1. Acquiring instance lists

Instance list request shall be multicasted (recipient IP=224.0.23.0) to search for devices in the network. The IP address and EOJs for each device can be learned using a response for the instance list.

```
Destination IP: 224.0.23.0
SEOJ: 0x05FF01
DEOJ: 0x0EF001
ESV:  0x62(GET)
OPC:  1
EPC:  0xD6
```

Ex.: When an air conditioner and a general lighting exist in the network

```
Controller                                      Air conditioner
Lighting
(IP=192.168.1.1)                            (IP=192.168.1.2)
(IP=192.168.1.3)
(EOJ=0x05FF01)                              (EOJ=0x013001)
(EOJ=0x029001)
    |  IP SRC:192.168.1.1, DST:224.0.23.0          |
|
    |  1081 0001 0EF001 0EF001 62 01 D6 00         | IP SRC:192.168.1.1,
DST:224.0.23.0           |
    | ---------------------------------------------> | 1081 0001 0EF001 0EF001 62 01
D6 00          |
    | ------------------------------------------------------------------------------
-------------> |
    |                                                |
|
    |  IP SRC:192.168.1.2, DST:192.168.1.1         |
|
    |  1081 0001 0EF001 0EF001 72 01 D6 04 01013001 |
|
    | <--------------------------------------------- |
|
    |                                                |
|
    |                                                | IP SRC:192.168.1.3,
DST:192.168.1.1          |
    |                                                | 1081 0001 0EF001 0EF001 72 01
D6 04 01029001 |
    | <------------------------------------------------------------------------------
-------------- |
```

2. Acquiring the specifications version of device objects

Device properties may vary by Appendix version. Therefore, it is necessary to acquire the property value of "standard version information" of the device in advance.

```
Destination IP: IP address of the device
SEOJ: 0x05FF01
DEOJ: EOJ of the device object
ESV:  0x62(GET)
OPC:  1
EPC:  0x82
```

Ex.

```
Controller                                      Air conditioner
(IP=192.168.1.1)                              (IP=192.168.1.2)
(EOJ=0x05FF01)                                (EOJ=0x013001)
    |  IP SRC:192.168.1.1, DST:192.168.1.2          |
    |  1081 0002 05FF01 013001 62 01 82 00          |
    | ----------------------------------------------> |
    |                                               |
    |  IP SRC:192.168.1.2, DST:192.168.1.1          |
    |  1081 0002 013001 05FF01 72 01 82 04 00004B00 |
    | <---------------------------------------------- |
```

3. Acquiring node profile identification number (EPC=0x83)
   The IP address and EOJ shall be used to identify the device object. Normally, IP address of a device is set by DHCP. Therefore, the IP address may be altered when turning on the device again or after a certain amount of time has elapsed. The node profile identification number (EPC=0x83), as a mandatory property shall be used to track correspondence between the IP address and the device. The identification number must be a unique value across ECHONET Lite devices. An identification number (EPC=0x83) is also defined for a device object, but not mandatory.

```
Destination IP: IP address of the device
SEOJ: 0x05FF01
DEOJ: EOJ of the device object
ESV:  0x62(GET)
OPC:  1
EPC:  0x83
```

Ex.

```
Controller                                      Air conditioner
(IP=192.168.1.1)                              (IP=192.168.1.2)
(EOJ=0x05FF01)                                (EOJ=0x013001)
    |  IP SRC:192.168.1.1, DST:192.168.1.2          |
    |  1081 0003 05FF01 013001 62 01 83 00          |
    | ----------------------------------------------> |
    |                                               |
    |  IP SRC:192.168.1.2, DST:192.168.1.1          |
    |  1081 0003 013001 05FF01 72 01 83 11 FE00006900010203040506070809 0A0B0C
    | <---------------------------------------------- |
```

4. Acquiring device object property maps (EPC=0x9D, 9E, 9F)
   When accessing properties other than mandatory properties, it is necessary to know which property is implemented by the target device. Therefore, Status change announcement property map (EPC=0x9D), Set property map (0x9E), and a Get property map (0x9F) shall be acquired. If the number of properties is 15 or less, the EPC values shall be listed in the property map. If the number is 16 or more, they shall be encoded into a bitmap (reference: APPENDIX Annex 1).

Request:

```
Destination IP: IP address of the device
SEOJ: 0x05FF01
DEOJ: EOJ of the device object
ESV:  0x62(Get)
OPC:  1
EPC:  0x9D, 0x9E, 0x9F
```

Response (when the number of EPCs is 15 or less):

```
ESV:  0x62(Get_Res)
EDT:  Byte 1: number of EPCs, byte 2 and later: EPC list
Example of EDT: 04808FB0B3
```

Response (when number of EPCs are 16 or more):

```
ESV:  0x62(Get_Res)
OPC:  17
EDT:  Byte 1: number of EPCs, byte 2 and later: bitmap that represents EPC (see the
table below)
Example of EDT: 160B0101090000000101010303030303
(EPC: 80,81,82,83,87,88,89,8A,8B,8C,8D,8E,8F,90,9A,9B,9C,9D,9E,9F,B0,B3)
```

bitmap of EPC (bit value 1:implemented, 0:not implemented)

byte\bit	bit-7	bit-6	bit-5	bit-4	bit-3	bit-2	bit-1	bit-0
byte-2	0xF0	0xE0	0xD0	0xC0	0xB0	0xA0	0x90	0x80
byte-3	0xF1	0xE1	0xD1	0xC1	0xB1	0xA1	0x91	0x81
byte-4	0xF2	0xE2	0xD2	0xC2	0xB2	0xA2	0x92	0x82
byte-5	0xF3	0xE3	0xD3	0xC3	0xB3	0xA3	0x93	0x83
byte-6	0xF4	0xE4	0xD4	0xC4	0xB4	0xA4	0x94	0x84
byte-7	0xF5	0xE5	0xD5	0xC5	0xB5	0xA5	0x95	0x85
byte-8	0xF6	0xE6	0xD6	0xC6	0xB6	0xA6	0x96	0x86
byte-9	0xF7	0xE7	0xD7	0xC7	0xB7	0xA7	0x97	0x87
byte-10	0xF8	0xE8	0xD8	0xC8	0xB8	0xA8	0x98	0x88
byte-11	0xF9	0xE9	0xD9	0xC9	0xB9	0xA9	0x99	0x89
byte-12	0xFA	0xEA	0xDA	0xCA	0xBA	0xAA	0x9A	0x8A
byte-13	0xFB	0xEB	0xDB	0xCB	0xBB	0xAB	0x9B	0x8B
byte-14	0xFC	0xEC	0xDC	0xCC	0xBC	0xAC	0x9C	0x8C
byte-15	0xFD	0xED	0xDD	0xCD	0xBD	0xAD	0x9D	0x8D
byte-16	0xFE	0xEE	0xDE	0xCE	0xBE	0xAE	0x9E	0x8E

byte\bit	bit-7	bit-6	bit-5	bit-4	bit-3	bit-2	bit-1	bit-0
byte-17	0xFF	0xEF	0xDF	0xCF	0xBF	0xAF	0x9F	0x8F

Ex.

```
Controller                                      Air conditioner
(IP=192.168.1.1)                              (IP=192.168.1.2)
(EOJ=0x05FF01)                                (EOJ=0x013001)
    |  Get Property map(Anno)                        |
    |  IP SRC:192.168.1.1, DST:192.168.1.2           |
    |  1081 0004 05FF01 013001 62 01 9D 00           |
    | ---------------------------------------------> |
    |                                                |
    |  IP SRC:192.168.1.2, DST:192.168.1.1           |
    |  1081 0004 013001 05FF01 72 01 9D 05 04808FA0B0
    | <--------------------------------------------- |
    |                                                |
    |  Get Property map(Set)                         |
    |  IP SRC:192.168.1.1, DST:192.168.1.2           |
    |  1081 0004 05FF01 013001 62 01 9E 00           |
    | ---------------------------------------------> |
    |                                                |
    |  IP SRC:192.168.1.2, DST:192.168.1.1           |
    |  1081 0004 013001 05FF01 72 01 9E 06 05808FA0B0B3
    | <--------------------------------------------- |
    |                                                |
    |  Get Property map(Get)                         |
    |  IP SRC:192.168.1.1, DST:192.168.1.2           |
    |  1081 0004 05FF01 013001 62 01 9F 00           |
    | ---------------------------------------------> |
    |                                                |
    |  IP SRC:192.168.1.2, DST:192.168.1.1           |
    |  1081 0004 013001 05FF01 72 01 9F 07 06808FA0B0B3BB
    | <--------------------------------------------- |
```

5. Acquiring manufacturer codes (EPC=0x83)
   If the manufacturer name for the device object is known, it may be used to identify the device. Acquiring a mandatory property, the manufacturer code (EPC=0x8A) can identify the manufacturer name.

```
Destination IP: IP address of the device
SEOJ: 0x05FF01
DEOJ: EOJ of the device object
ESV:  0x62(GET)
OPC:  1
EPC:  0x8A
```

Ex.

```
Controller                                      Air conditioner
(IP=192.168.1.1)                              (IP=192.168.1.2)
(EOJ=0x05FF01)                                (EOJ=0x013001)
    |  IP SRC:192.168.1.1, DST:192.168.1.2           |
```

```
|  1081 0004 05FF01 013001 62 01 8A 00          |
| ------------------------------------------> |
|                                               |
|  IP SRC:192.168.1.2, DST:192.168.1.1          |
|  1081 0004 013001 05FF01 72 01 8A 03 000068   |
| <------------------------------------------ |
```

Manufacturer codes consist of 3 bytes. The table showing the corresponding manufacturer codes and manufacturer names shall be disclosed only to ECHONET Consortium members. Acquiring manufacturer codes from actual devices can clarify the correspondence with manufacturer names. Therefore, Chapter 6 shows a table explaining correspondences between the manufacturer codes of devices owned by the Kanagawa Institute of Technology and manufacturer names.

2.8 Error processing

The following explains error processing when receiving request messages(ESV=0x60, 61, 62, 63, 6E), (reference: Part 2, 4.2.2, Part 2, Annex 1, System Guideline 2.2). Depending on conditions, one of followings will be processed.

- Send no response and ignore the operation.
- Do the acceptance process of the message(*), and ignore the operation.
- Reply SNA and ignore the operation.

(*)In case of ESV=0x60, "reply no response", in case of ESV=0x61, "response ESV=0x71" and in case of ESV=0x6E, "response ESV=0x7E.

2.8.1 Message frame error

Condition: In case a message does not comply with the ECHONET Lite frame format.
Process: Send no response and ignore the operation
Note: SetGet service utilizes different frame structure.

2.8.2 EHD error

Condition: In case EHD is not 0x1081.
Process: Send no response and ignore the operation.

2.8.3 DEOJ error

Condition: In case a specified object by the DEOJ does not exist.
Process: Send no response and ignore the operation
Note: Please note the case when instance code is 0x00. (reference: Part 2, Annex 1)

2.8.4 ESV error

Condition: In case the value of ESV does not comply with the specifications.
Process: Send no response and ignore the operation
(reference: Part 2, Annex 1)

2.8.5 EPC error

Condition: In case the specified EPC is not implemented to the specified ESV.
Process: Reply SNA and ignore the operation.
(reference: Part 2, Annex 1)

2.8.6 EDT error

Condition: In case the data size of EDT does not comply with the specifications.
Process: One of followings (A): Send no response and ignore the operation. (B): Reply SNA and ignore the operation. (reference: Part 2, Annex 1)

Condition: In case the value of EDT does not comply with the specifications. (ex: out of range, the value represents no status.)
Process(recommended): Do the acceptance process of the message(*) and ignore the operation.
Process(acceptable): Reply SNA and ignore the operation.
(*)In case of ESV=0x60 "reply no response", in case of ESV=0x61 "response ESV=0x71" and in case of ESV=0x6E "response ESV=0x7E.
(reference:System Guideline 2.2)

Example of responding Get_SNA (respond with PDC=0x00)

```
Request: controller (0x05FF01) -> air conditioner (0x013001)
10 81 00 01 05 FF 01 01 30 01 62  01  A2  00
<---> <---> <------> <------> <-> <-> <-> <->
 EHD   TID    SEOJ     DEOJ   ESV OPC EPC PDC

Response: air conditioner (0x013001) -> controller (0x05FF01)
10 81 00 01 01 30 01 05 FF 01 52  01  A2  00
<---> <---> <------> <------> <-> <-> <-> <->
 EHD   TID    SEOJ     DEOJ   ESV OPC EPC PDC
```

Example of responding SetC_SNA (respond SetC EDT value)

```
Request: controller (0x05FF01) -> air conditioner (0x013001)
10 81 00 01 05 FF 01 01 30 01 61  01  80  01  32
<---> <---> <------> <------> <-> <-> <-> <-> <->
 EHD   TID    SEOJ     DEOJ   ESV OPC EPC PDC EDT

Response: air conditioner (0x013001) -> controller (0x05FF01)
10 81 00 01 01 30 01 05 FF 01 51  01  80  01  32
<---> <---> <------> <------> <-> <-> <-> <-> <->
 EHD   TID    SEOJ     DEOJ   ESV OPC EPC PDC EDT
```

2.9 Property values and implementation

In case EDT is a status

Devices do not always implement all statuses defined in the specifications. Devices respond normally to Set requests that are not implemented, but ignore it for the internal processing(reference: system design guideline 2.1(3)).
Example: EPC=0xB0 (operation mode) for air conditioners are defined as 0x41 (auto), 0x42 (cooling), 0x43 (heating), 0x44 (dehumidifying), 0x45 (ventilation), and 0x40 (other). When cooling-only air conditioners receive a message (ESV=SetC, EPC=0xB0, EDT=0x43), it responds Set_Res, but does not alter the inner operation mode. Property values shall not be changed.

In case EDT is a level

There are many properties that designate high-low settings with eight-step levels (0x31 through 0x38), including EPC=0xA0 (air flow rate setting) for air conditioners. For example, if there are only three steps (low, medium, and high) internally for the airflow settings, it is recommended that you map all EDT levels to any one of the settings of the actual device (reference: system design guideline 2.1(2)).

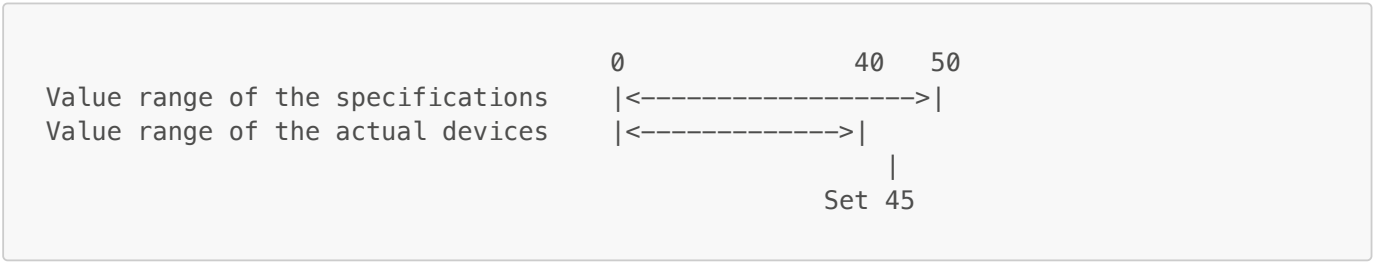
Examples of EDT values and inner setting mapping

EDT value	0x31	0x32	0x33	0x34	0x35	0x36	0x37	0x38
Inner setting	Low	Low	Medium	Medium	Medium	Medium	High	High

When EDT=0x33 is SetC, the airflow setting is internally “medium”.
For the Get operation, the following two implementation patterns can be considered.
(A) Set value is responded as is
(B) A representative value corresponding with the inner setting (low:0x31, medium:0x35, high:0x38) shall be responded (reference: system design guideline 2.1(2))
In the case of (B), the values of set EDT and got EDT may not be same, such as Set (EDT=0x34) -> Get (EDT=0x35).

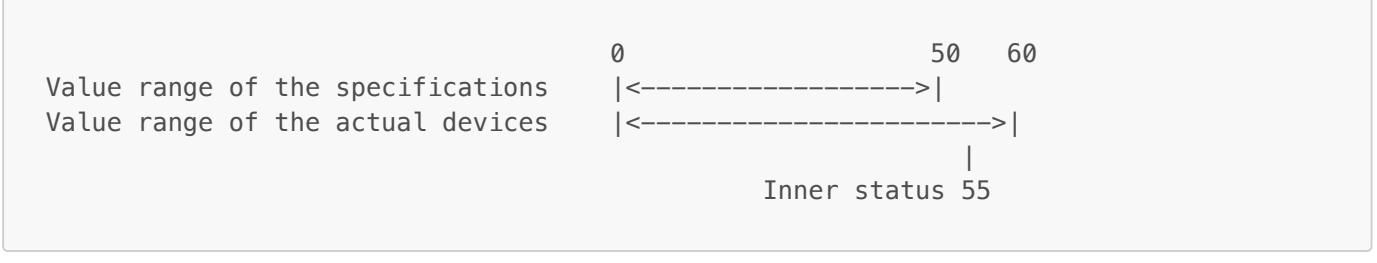
In case EDT is a numerical value (when value range of the actual device is within the value range of the specifications)

If a Set request of values not implemented is received when the value range of the actual device is within the value range of the specifications, values are at the edge of the implemented values (reference: system design guideline 2.1(1)). For example, as shown in the below figure, if SetC (EDT=45) is received when the maximum value of the specification value range is 50 and the maximum value of the implementation value range is 40, the property value shall be 40.



In case EDT is a numerical value (when value range of actual device exceeds the value range of the specifications)

When the actual device value range extends beyond the specification value range, and if a Get request is received when the inner status extends beyond the specification value range, values corresponding with overflow or underflow shall be responded (reference: Part 2, 6.2.2(2)). For example, as shown in the below figure, if a Get specification is received when the maximum value of the specification value range is 50 and the maximum value of the implementation value range is 60, the value 0xFF corresponding with the overflow shall be responded.



2.10 Process of multiple operations

A set of EPC, PDC, and EDT shall be called “operation”. Two or more operations can be described in an ECHONET Lite message. For example, a read request for three properties (EPC=0x80, 0x81, and 0x82) can be sent at once with ESV=0x62 (Get). The number of operations to be sent at once shall be set at OPC (reference: Part 2, 4.2.3). Since error processing is complicated, implementation of only OPC=0x01 at the time of prototyping is acceptable. However, AIF certification has some tests that require sending/receiving of multiple OPC messages.

Get: Normal process for multiple operations

A set of EPC and PDC(0x00) is repeated by the number of OPC in a request message.
A set of EPC, PDC and EDT is repeated by the number of OPC in a response message.
The following is an example in case of OPC=0x03.

```
Request: controller (0x05FF01) -> air conditioner (0x013001)
10 81 00 01 05 FF 01 01 30 01 62  03  80   00   81   00   82   00
<---> <---> <------> <------> <-> <-> <--> <--> <--> <--> <--> <-->
 EHD   TID    SEOJ     DEOJ   ESV OPC EPC1 PDC1 EPC2 PDC2 EPC3 PDC3

Response: air conditioner (0x013001) -> controller (0x05FF01)
10 81 00 01 01 30 01 05 FF 01 72  03  80   01   31   81   01   00   82   04
00004400
<---> <---> <------> <------> <-> <-> <--> <--> <--> <--> <--> <--> <--> <--> <-----
->
 EHD   TID    SEOJ     DEOJ   ESV OPC EPC1 PDC1 EDT1 EPC2 PDC2 EDT2 EPC3 PDC3 EDT3
```

Get: Abnormal process for multiple operations

If there is a failure to process even one of the operations, ESV shall be 0x52 (Get_SNA), and the EPC that failed to be processed shall be responded as PDC=0x00.
The followings is an example that the third operation is abnormal processing.

```
Request: controller (0x05FF01) -> air conditioner (0x013001)
10 81 00 01 05 FF 01 01 30 01 62  03  80   00   81   00   A2   00
<---> <---> <------> <------> <-> <-> <--> <--> <--> <--> <--> <-->
 EHD   TID    SEOJ     DEOJ   ESV OPC EPC1 PDC1 EPC2 PDC2 EPC3 PDC3

Response: air conditioner (0x013001) -> controller (0x05FF01)
10 81 00 01 01 30 01 05 FF 01 52  03  80   01   31   81   01   00   A2   00
<---> <---> <------> <------> <-> <-> <--> <--> <--> <--> <--> <--> <--> <-->
 EHD   TID    SEOJ     DEOJ   ESV OPC EPC1 PDC1 EDT1 EPC2 PDC2 EDT2 EPC3 PDC3
```

Set: Normal process for multiple operations

In case of ESV=0x60(SetI), if all of the operations are acceptable, no response is required.
The following is an example in case of ESV=0x60(SetC) and OPC=0x02.

```
Request: controller (0x05FF01) -> air conditioner (0x013001)
10 81 00 01 05 FF 01 01 30 01 61  02  80   01   30   81   01   08
<---> <---> <------> <------> <-> <-> <--> <--> <--> <--> <--> <-->
 EHD   TID    SEOJ     DEOJ   ESV OPC EPC1 PDC1 EDT1 EPC2 PDC2 EDT2

Response: air conditioner (0x013001) -> controller (0x05FF01)
10 81 00 01 01 30 01 05 FF 01 71  02  80   00   81   00
```

```
<---> <---> <------> <------> <-> <-> <--> <--> <--> <-->
 EHD   TID    SEOJ     DEOJ   ESV OPC EPC1 PDC1 EPC2 PDC2
```

Set: Abnormal process for multiple operations

If failing to process even one of the operations, ESV of the response shall be 0x50 (SetI_SNA) or 0x51 (SetC_SNA) respectively.
Response data shall be PDC=0x00 for the properly processed operations, and the same PDC and EDT data of the request message for the operations that can not be processed.
The following is an example that the first operation is abnormal processing.

```
Request: controller (0x05FF01) -> air conditioner (0x013001)
10 81 00 01 05 FF 01 01 30 01 61  02  80   01   32   81   01   08
<---> <---> <------> <------> <-> <-> <--> <--> <--> <--> <--> <-->
 EHD   TID    SEOJ     DEOJ   ESV OPC EPC1 PDC1 EDT1 EPC2 PDC2 EDT2

Response: air conditioner (0x013001) -> controller (0x05FF01)
10 81 00 01 01 30 01 05 FF 01 51  02  80   01   32   81   00
<---> <---> <------> <------> <-> <-> <--> <--> <--> <--> <-->
 EHD   TID    SEOJ     DEOJ   ESV OPC EPC1 PDC1 EDT1 EPC2 PDC2
```

2.11 Smart electric energy meters

The following describes important notes related to low-voltage smart electric energy meters (EOJ=0x0288) and high-voltage smart electric energy meters (EOJ=0x028A).

Low-voltage smart electric energy meters (EOJ=0x0288)

- The physical layer is Wi-SUN (wireless) or G3-PLC (wired)
- One-to-one connection with IPv6
- PANA certification

High-voltage smart electric energy meters (EOJ=0x028A)

- The physical layer is wired LAN
- IPv6

3. Advanced level

3.1 SetGet

Using ESV=0x6E (SetGet) can send Set and Get operations in one message (reference: Part 2, 4.2.3.4). However, the frame configuration of the message becomes different from others. Error processing is very complicated. Implementation of the SetGet process is optional. If you are not implementing it, SetGet_SNA (OPCSet=0, OPCGet=0) shall be responded.

When the same EPC is designated by Set and Get operation in a message with SetGet service, which of Set or Get should be executed first shall depend on the implementation (reference: Part 2, 4.2.3.4). On the other hand, the system design guideline defines that Get process shall be executed after executing the Set process as an implementation policy. (Reference: system design guideline 2.6).

Message frame configuration

SetC operation (EPC, PDC, EDT) shall be described after OPC_Set, and then, Get operation(EPC, PDC) shall be described after OPC_Get.

```
Request
+---+---+----+----+----+------+----+----+----+---+----+----+----+------+----+----+--
-+----+----+
|EHD|TID|SEOJ|DEOJ|ESV
|OPCSet|EPC1|PDC1|EDT1|...|EPCm|PDCm|EDTm|OPCGet|EPC1|PDC1|...|EPCn|PDCn|
|   |   |    |    |0x6E|  (m) |    |    |    |   |    |    |    |  (n) |    |0x00|
|    |0x00|
+---+---+----+----+----+------+----+----+----+---+----+----+----+------+----+----+--
-+----+----+

Response
+---+---+----+----+----+------+----+----+---+----+----+------+----+----+----+---+---
-+----+----+
|EHD|TID|SEOJ|DEOJ|ESV
|OPCSet|EPC1|PDC1|...|EPCm|PDCm|OPCGet|EPC1|PDC1|EDT1|...|EPCn|PDCn|EDTn|
|   |   |    |    |0x7E|  (m) |    |0x00|   |    |0x00|  (n) |    |    |    |   |
|    |    |
+---+---+----+----+----+------+----+----+---+----+----+------+----+----+----+---+---
-+----+----+
```

Example of normal processing: OPC_Set=2, OPC_Get=2

Response: PDC=0x00 for Set, while property value for Get

```
Request: controller (0x05FF01) -> air conditioner (0x013001)
10 81 00 01 05 FF 01 01 30 01 6E  02     80   01   30   81   01   08   02     A0
00   B0   00
<---> <---> <------> <------> <-> <----> <--> <--> <--> <--> <--> <--> <----> <-->
<--> <--> <-->
 EHD   TID    SEOJ     DEOJ   ESV OPCSet EPC1 PDC1 EDT1 EPC2 PDC2 EDT2 OPCGet EPC1
PDC1 EPC2 PDC2

Response: air conditioner  (0x013001) -> controller  (0x05FF01)
10 81 00 01 01 30 01 05 FF 01 7E  02     80   00   81   00   02     A0   01   41
B0   01   42
<---> <---> <------> <------> <-> <----> <--> <--> <--> <--> <----> <--> <--> <-->
<--> <--> <-->
 EHD   TID    SEOJ     DEOJ   ESV OPCSet EPC1 PDC1 EPC2 PDC2 OPCGet EPC1 PDC1 EDT1
EPC2 PDC2 EDT2
```

Example of abnormal processing: OPC_Set=2, OPC_Get=2

If failing to process even one of the operations, ESV shall be 0x5E (SetGet_SNA), and if failing to process Set, Set EDT value shall be responded, while failing to process Get, PDC=0x00 shall be responded. The followings are the example that the first operation of Set and the second operation of Get are abnormal processing.

```
Request: controller (0x05FF01) -> air conditioner (0x013001)
10 81 00 01 05 FF 01 01 30 01 6E  02     80   01   30   81   01   08   02     A0
00   B0   00
<---> <---> <------> <------> <-> <----> <--> <--> <--> <--> <--> <--> <----> <-->
<--> <--> <-->
 EHD   TID    SEOJ     DEOJ   ESV OPCSet EPC1 PDC1 EDT1 EPC2 PDC2 EDT2 OPCGet EPC1
PDC1 EPC2 PDC2

Response: air conditioner  (0x013001) -> controller  (0x05FF01)
10 81 00 01 01 30 01 05 FF 01 5E  02     80   01   30   81   00   02     A0   01
41   B0   00
<---> <---> <------> <------> <-> <----> <--> <--> <--> <--> <--> <----> <--> <-->
<--> <--> <-->
 EHD   TID    SEOJ     DEOJ   ESV OPCSet EPC1 PDC1 EDT1 EPC2 PDC2 OPCGet EPC1 PDC1
EDT1 EPC2 PDC2
```

In case SetGet service is not implemented

Implementation of the SetGet service is optional. If the service is not implemented, SetGet_SNA (OPCSet=0, OPCGet=0) shall be responded to SetGet.

```
Request: controller (0x05FF01) -> air conditioner (0x013001)
10 81 00 01 05 FF 01 01 30 01 6E  02     80   01   30   81   01   08   02     A0
00   B0   00
<---> <---> <------> <------> <-> <----> <--> <--> <--> <--> <--> <--> <----> <-->
<--> <--> <-->
 EHD   TID    SEOJ     DEOJ   ESV OPCSet EPC1 PDC1 EDT1 EPC2 PDC2 EDT2 OPCGet EPC1
PDC1 EPC2 PDC2

Response: air conditioner  (0x013001) -> controller  (0x05FF01)
10 81 00 01 01 30 01 05 FF 01 5E  00     00
<---> <---> <------> <------> <-> <----> <---->
 EHD   TID    SEOJ     DEOJ   ESV OPCSet OPCGet
```

3.2 Implementation in a real world

Implementation of ECHONET Lite in the real world is described in this chapter for prototyping experimental products and for producing marketable products. For prototyping, implementation shall be least required to check functionalities. For producing marketable products, certification shall be considered. Regarding certifications, see [Chapter 4] (#Chap4)

3.2.1 Controllers (for prototyping)

Basically, controllers do not need to respond to receiving Get or SetC commands that are received, so implementation is not mandatory at this time. However, there are some devices that check the nature of controllers, so in that case, it is recommended to respond when receiving Get commands to the following EPCs.

Node profile

EPC	property name	EDT value (example)
0x82	Version information	0x010D0100: V1.13
0x83	Identification number	0xFEFFFFFF000102030405060708090A0B0C
0x8A	Manufacturer code	0xFFFFFF: Experimental codes (reference: Appendix 2.6)

Super class

EPC	property name	EDT value (example)
0x80	Operation status	0x30: ON
0x8A	Manufacturer code	0xFFFFFF: Experimental codes (reference: Appendix 2.6)

A controller will send commands with OPC=1 using Get and SetC services.

3.2.2 Devices (for prototyping)

If a controller is a product, mandatory properties for a node profile and device objects must be implemented.
EPC=0x83(Identification number) of a node profile consists of 0xFE(1 byte), manufacturer code(3 bytes) and unique ID (13 bytes). The following value is a example of an experimental manufacturer code (0xFFFFFF) and a unique ID(000102030405060708090A0B0C). This value is acceptable if this device is the only one used in the network. However, if there is possibility of using two or more devices, identification numbers for these devices must be unique values(*). Using the same value may lead malfunction of the controller.

(*) The value shall be changed manually or a unique value shall be generated using random numbers or a MAC address.

Examples of property values:

Node profile

EPC	property name	EDT value (example)
0x80	Operating status	0x30:ON
0x82	Version information	0x010D0100: V1.13
0x83	Identification number	0xFEFFFFFF000102030405060708090A0B0C
0x8A	Manufacturer code	0xFFFFFF: Experimental codes (reference: Appendix 2.6)
0x9D	Status change announcement property map	0x0280D5
0x9E	Set property map	0x00
0x9F	Get property map	0x0B8082838A9D9E9FD3D5D6D7
0xD3	Number of self-node instances	0x000001
0xD4	Number of self-node classes	0x0002
0xD5	Instance list notification	0x01013001
0xD6	Self-node instance list S	0x01013001
0xD7	Self-node class list S	0x010130

Super class of device objects

EPC	property name	EDT value (example)
0x80	Operation status	0x30: ON
0x81	Installation location	0x00: Not set
0x82	Standard version information	0x00004C00: L
0x88	Fault status	0x42: No fault
0x8A	Manufacturer code	0xFFFFFF: Experimental codes (reference: Appendix 2.6)
0x9D	Status change announcement property map	in accordance with actual implementation
0x9E	Set property map	in accordance with actual implementation
0x9F	Get property map	in accordance with actual implementation

The implementation corresponds with Get and SetC out of the services that are received by devices, and respond Get_Res and Set_Res. The OPC only corresponds with 1. No particular error processing is needed. EPC=0xD5 INF of the startup process node profile shall be implemented.

3.2.3 Controllers (for producing a marketable product)

- Mandatory properties of a node profile object and device objects must be implemented.
- EPC=0xBF(Unique identifier data) of a node profile object shall not be implemented(*).
- The property value of EPC=0x81(installation location) of device objects is defined with 1 byte and 17 bytes. However, 1-byte implementation is more commonly used.
- EPC=0x83(Identification number) of device objects shall be implemented, although it is not mandatory.
- ESVs to be used for sending requests are 0x61(SetC) and 0x62(Get). SetI, INF_REQ, and SetGet shall not be used.
- The ESV to be used for sending notifications is 0x73(INF). 0x74(INFC) shall not be used.
- There are some properties that take time to send a response after receiving Get request. Historical values of low-voltage smart electric energy meters are an example. Time-out values should be taken into considerations by property.
- It is recommended to Get a property value to check Set operation after sending SetC. Note that the Set value and Get value may be different.
    - The value stays same
        - In case completion of a Set operation takes time
        - In case the device does not implement the set value
    - The value changes but not identical to the value of Set
        - Depending on implementation by the device
- Receiving requests with ESV(0x60, 0x61, 0x62, 0x63, 0x6E) shall be correspond
- Once an INFC(0x74) is received, INFC_Res(0x7A) shall be responded regardless of the EPC and EDT value.
- If SetGet is received, SetGet_SNA (OPCSet=0, OPCGet=0) shall be responded.
- Please confirm applicable AIF certification specifications to determine whether a send message needs to support multiple OPCs.

(*) It was mandatory through ECHONET Lite V1.01. It was originally specified to identify the node profile. However, due to over-complicated specifications, it was used for EPC=0x83 to identify the node profile in V1.1 or later.

3.2.4 Devices (for producing a marketable product)

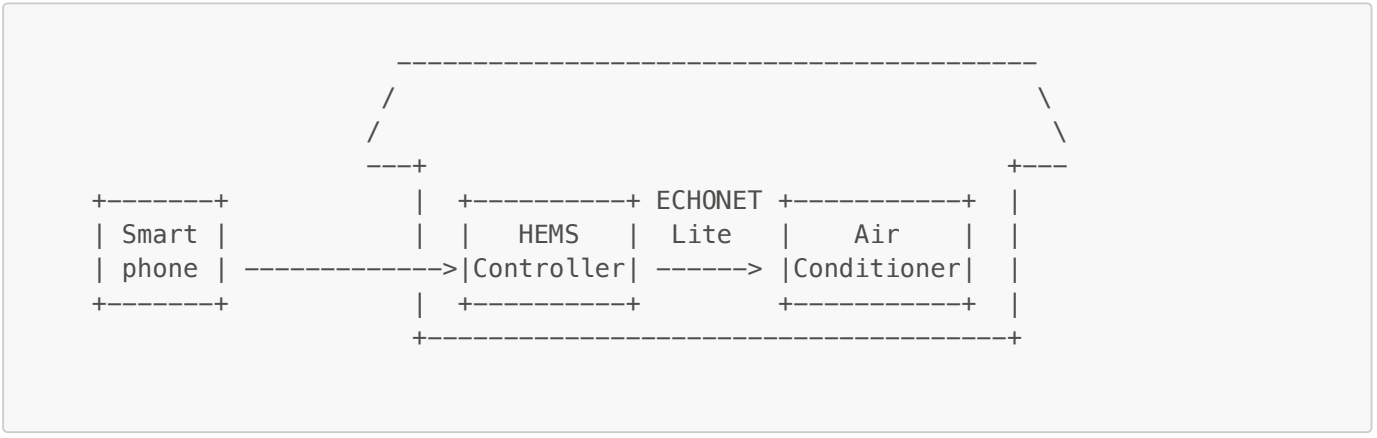
- Mandatory properties of a node profile object and device objects must be implemented.

- EPC=0xBF(Unique identifier data) of a node profile object shall not be implemented(*).
- The property value of EPC=0x81(installation location) of device objects is defined with 1 byte and 17 bytes. However, 1-byte implementation is more commonly used.
- Implementation of device object EPC=0x83(Identification number) is recommended, although they are not mandatory (reference: system design guideline 2.8).
- The ESV to be used for sending notifications is 0x73(INF). 0x74(INFC) shall not be used.
- Receiving requests with ESV(0x60, 0x61, 0x62, 0x63, 0x6E) shall be correspond
- Once an INFC (0x74) is received, INFC_Res(0x7A) shall be responded regardless of the EPC and EDT value.
- If SetGet is received, SetGet_SNA (OPCSet=0, OPCGet=0) shall be responded.
- Please confirm applicable AIF certification specifications to determine whether a receive message needs to support multiple OPCs.

(*) It was mandatory through ECHONET Lite V1.01. It was originally specified to identify the node profile. However, due to over-complicated specifications, it was used for EPC=0x83 to identify the node profile in V1.1 or later.

3.3 Remote control setting

EPC=0x93(Remote control setting) of the device object super class relates to remote controls under the Electrical Appliance and Material Safety Act(reference: APPENDIX2.16, system design guideline Chapter 5). For example, in cases where an air conditioner in a house is controlled from outside of the house using a smartphone (see figure below), the system configuration assumed is one that uses ECHONET Lite for communication between the HEMS Controller and the air conditioner. Note that EPC=0x93 is not a mandatory property. The following explains “cases implementing remote control settings”.



Implementation by a controller

When a controller receives a control from outside the house (e.g. the internet) and controls the devices in the house with ECHONET Lite, an operation of EPC=0x93, EDT=0x42(through a public network)” shall be added as the first operation. Following is an example of controlling ON/OFF of an air-conditioner remotely.

> 1081 0001 05FF01 013001 61 02 930142 800130

- In case of Get operation by sending a command from outside the house, an operation with EPC-0x93 shall not be added.
- In case of in-house control(e.g. operation by controller GUI), an operation with EPC-0x93 shall not be added.

Implementation by a device

- In case a received Set command has an operation of EPC=0x93, EDT=0x42(through a public network)
    - Set the value of 0x42 to EPC=0x93.

    - Operations shall be performed based on the determination of operation by the device (this shall be at the installer's own responsibility, since this matter involves interpretation of the law).

- In case received Set command does not have an operation with EPC=0x93
    - Set the value of 0x41(not through a public network) to EPC=0x93.
    - Process Set operation.
- In case operated with a device panel or a remote controller.
    - Set the value of 0x41(not through a public network) to EPC=0x93.

3.4 Transmission-only nodes

Handling of transmission-only devices is defined mainly assuming battery-driven sensors.

- The profile node object EOJ for a transmission-only node is 0x0EF002 (reference: Part 2, 4.3.1)
- The profile node object instance list (EPC=0xD6) shall be regularly multicasted with INF (reference: system design guideline 2.10)
- Implementation of mandatory properties are not required (reference: Part 2, 6.2.3).

3.5 Differences between ECHONET Lite versions

The following version of ECHONET Lite standards re published as of August 2019.

> Version 1.00, 1.01, 1.10, 1.11, 1.12, 1.13

The following table shows important information between different versions.

Version	Notes
1.00 -> 1.01	Nothng special
1.01 -> 1.10	- Access rule of EPC=0xBF(unique identifier data) of node profile object V1.00 and V1.01: mandatory V1.10: not mandatory (reference:part2 6.11.1 table6-7. - Data format of EPC=0xB3(Identification number) of node profile object V1.00 and V1.01: 9 or 17bytes V1.10: 17bytes - Device search by controllers Typical method at V1.00 and V1.01: Multicasting INF_REQ of EPC=0xD5(Instance list) to node profile object Recommended method at V1.10: Multicasting Get of EPC=0xD6(Instance list S) to node profile object. - The value of DEOJ of node profile for the transmission-only node V1.00 and V1.01: No specific info. Use V1.10: 0x0EF002(reference: Part 2, 6.11.1).
1.10 -> 1.11	- Added guidelines about operations via WAN(reference: Part 5, Chapter 6)。
1.11 -> 1.12	Nothng special
1.12 -> 1.13	- Separated part 5 to ECHONET Lite system guidelines.

4. Certification systems

4.1 ECHNONET consortium

The ECHONET Consortium is an organization that develops ECHONET Lite standards. International standarizaton and establishing certification system are also within main activities. ECHONET Lite protocol is open and can be used free of charge, however only members can get certification. There are two Certification systems, ECHONET Lite certification system and ECHONET Lite AIF certification system. Documents about certifications are disclosed to members only.

4.2 ECHONET Lite certification system

The ECHNONET Lite standards certification verifies proper implementation of the common items under the ECHONET Lite specifications for all devices. The Consortium shall provide no testing tools. The test items are as follows:

- Access to node profile properties
- Access to properties of device object superclass
- Instance list notification when startup

4.3 ECHONET Lite AIF certification system

AIF specification certification verifies the values of device object-specific properties by accessing the properties. The Consortium provides testing tools to members. As of August 2019, devices specified in the following categories and controllers that control the applicable devices should obtain AIF specification certification after acquiring ECHNONET Lite standards certification.

- Low/high-voltage smart electric energy meters
- Home air conditioners
- Household solar power generations
- Instantaneous water heaters
- Lightings
- Storage batteries
- Electric vehicle chargers/dischargers, electric vehicle chargers
- Electric vehicle chargers/dischargers
- Fuel cells
- Commercial-use package air conditioner
- Refrigerated display cases
- Lighting system
- Extended lighting system

Note that some device types have additional mandatory properties to be implemented according to the AIF specifications. Note that there are some test items that assume multiple OPCs.

Annex. Manufacturer code

Table showing correspondences between manufacturer codes and manufacturer names for devices owned by the Kanagawa Institute of Technology

Manufacture code	Manufacture name
0x000005	Sharp Corporation
0x000006	Mitsubishi Electric Corporation
0x000008	Daikin Industries, Ltd.
0x000009	NEC Corporation
0x00000B	Panasonic Corporation

Manufacture code	Manufacture name
0x000016	Toshiba Corporation
0x00001B	Toshiba Lighting and Technology Corporation
0x000022	Hitachi Appliances, Inc.
0x00004E	Fujitsu Ltd.
0x000053	Ubiquitous Corporation
0x000053	Rinnai Corporation
0x00005E	GM Solar Corporation
0x000060	Sony Computer Science Laboratories, Inc.
0x000063	Kawamura Electric, Inc.
0x000064	Omron Corporation, Inc.
0x000069	Toshiba Litestyle Products & Services Corporation
0x00006C	Nichikon Corporation
0x00006F	Buffalo Inc.
0x000072	ENERES Co., Ltd.
0x000077	Kanagawa Institute of Technology
0x000078	Hitachi Maxell Ltd.
0x00007D	POWERTECH INDUSTRIAL
0x000086	Nippon Telegraph and Telephone West Corporation
0x00008A	Fujitsu General Ltd.
0x0000A3	Chubu Electric Power Co., Ltd.
0x0000AA	Ten Feet Wright, Inc.
0x0000AE	Shikoku Electric Power Co., Inc.
0x0000B5	Chugoku Electric Power Co., Inc.
0x0000B6	Bunka Shutter Co., Ltd.
0x0000B8	Hokkaido Electric Power Co., Inc.